

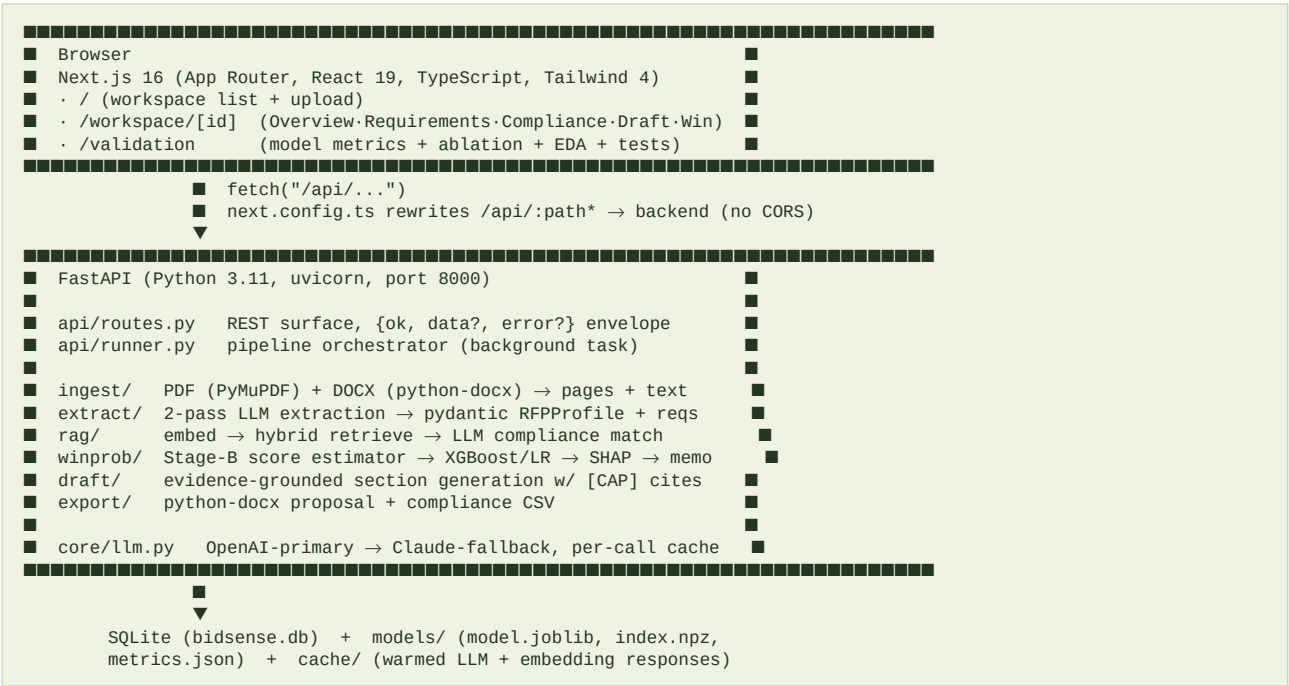
BidSense — Technical Architecture & Stack

AI-Powered Bid & Proposal Response Engine · CUST Hackathon 2026 (Problem #1)

This is the engineering deep-dive. For the judge-facing overview see ../README.md; for the user-facing walkthrough of the workspace tabs see workflow-and-tabs.md.

1. System overview

BidSense is a two-tier application: a **Next.js 16** frontend and a **FastAPI** Python backend, with **SQLite** for persistence and a **disk-cached dual-provider LLM layer** that lets the entire demo run offline once warmed.



Design priorities (rubric-mapped): real trained ML with explanations (not an LLM guessing a number), every drafted claim traceable to a real record, and full offline resilience for an unreliable venue network.

2. Technology stack

Frontend

Layer	Choice	Version	Why
Framework	Next.js (App Router)	16.1.6	Server components, file routing, built-in API rewrites
UI runtime	React	19.2.3	Latest stable, concurrent features
Language	TypeScript	5.x	Type-safe API contracts (lib/types.ts)

Layer	Choice	Version	Why
Styling	Tailwind CSS	4.x	Token-based @theme; light/dark via one <code>html.dark</code> override
Icons	lucide-react	0.577	Consistent SVG icon set (no emoji icons)
Charts	recharts + hand-rolled SVG	3.3	Gauge, SHAP bars, score-win curve
Utilities	clsx, tailwind-merge, class-variance-authority	—	Conditional class composition

The frontend ships **no API keys and no business logic** — it is a typed client over the backend REST surface. All network calls go through `lib/api.ts`, which unwraps the `{ok, data}` envelope and throws on `ok: false`.

Backend

Layer	Choice	Version	Why
Web framework	FastAPI	0.115	Async, pydantic-native, auto OpenAPI docs at <code>/docs</code>
Server	uvicorn[standard]	0.32	ASGI server
Validation	pydantic	2.9	Schemas are the single source of truth for extracted data
Persistence	SQLite (stdlib <code>sqlite3</code>)	—	Zero-setup, per-call connections, thread-safe for background tasks
Uploads	python-multipart	≥0.0.9	Multipart file ingest

ML / data

Layer	Choice	Version	Why
Gradient boosting	XGBoost	2.1.1	Primary win-probability candidate
Classic ML + CV	scikit-learn	1.5.2	Logistic-regression benchmark, StratifiedKfold, metrics
Explainability	SHAP	≥0.46	Per-feature contributions (TreeExplainer / LinearExplainer)
Dataframes	pandas	2.2.3	History/feature engineering
Numerics	numpy	≥1.26	Vectors, embedding math
Model IO	joblib	1.4.2	Persist the selected model bundle

Document & LLM

Layer	Choice	Version	Why
PDF parsing	PyMuPDF (fitz)	≥1.24	Fast, page-accurate text extraction
DOCX parsing/writing	python-docx	≥1.1	Read tenders, write the proposal export
LLM (primary)	OpenAI	sdk ≥1.50	Extraction, matching, drafting, memo, embeddings
LLM (fallback)	Anthropic Claude	sdk ≥0.40	Automatic failover on any OpenAI error
Config	python-dotenv	1.0.1	<code>.env</code> -driven keys and model names

Layer	Choice	Version	Why
Tests	pytest + httpx	8.3 / 0.27	Unit + retrieval + model-metric gates

Default models are configurable in `core/config.py`: `CLAUDE_MODEL=claude-sonnet-4-6`,
`OPENAI_MODEL=gpt-5.2`, `EMBEDDING_MODEL=text-embedding-3-small`.

3. Request flow & the no-CORS proxy

The browser only ever talks to its own origin. `next.config.ts` rewrites every `/api/:path*` to the FastAPI host:

```
async rewrites() {
  return [{ source: "/api/:path*", destination: `${BACKEND}/api/:path*` }];
}
```

So `fetch("/api/workspaces")` from the client is transparently served by `http://localhost:8000/api/workspaces`. No CORS preflight in the hot path, and the backend URL is swappable per environment via `BACKEND_URL`. (The backend also keeps a permissive CORS middleware as a fallback for direct calls.)

Every backend response uses one envelope:

```
{ "ok": true, "data": { ... } }
{ "ok": false, "detail": "Workspace not found" }
```

4. The LLM layer — dual provider + disk cache (offline replay)

`core/llm.py` is the single choke-point for every model call. Three properties make it demo-safe:

1. **Provider failover.** `complete_json()` / `complete_text()` try **OpenAI first**, and on *any* exception fall back to **Claude** with the same prompt. If both fail, a `RuntimeError` lists both errors. Downstream callers (matcher, drafter, memo) additionally degrade to deterministic logic so the pipeline never hard-stops.
2. **Per-call disk cache.** Each call is keyed by `sha256(system + user)[:32]` and written to `cache/<bucket>/<key>.json` with the provider that answered. A warmed cache means the full three-document demo **replays with the network disabled and no API keys** — the headline resilience feature. Buckets in use: `extract-meta`, `extract-reqs`, `match`, `draft`, `memo`, `embeddings`.
3. **Robust JSON parsing.** `parse_json_response()` strips `` `json`` fences and, if needed, walks the first balanced `{...}/[...]` block — LLMs that wrap or prepend prose still parse.

Embeddings — dual vector space

`embed()` returns `(matrix, space)` where `space` is `"openai"` or `"hash"`:

- Online: OpenAI `text-embedding-3-small` (1536-d), cached per text.
- Offline: a deterministic **hashing-trick embedding** (`hash_embed`, 512-d, unigrams + bigrams, signed buckets, L2-normalized).

The capability index is built in **both** spaces at seed time (`rag/embeddings.py` → `capability_index.npz`), so retrieval works regardless of whether the query could reach OpenAI. This is why offline mode degrades

gracefully instead of breaking.

5. The pipeline (orchestration)

`api/runner.py` runs four stages as a FastAPI **background task**, persisting a per-step progress record so the UI can render a live stepper. Steps: `extract` → `match` → `winprob` → `draft`.

```
POST /api/workspaces/{id}/run
status: extracting → matching → scoring → drafting → ready
each step: set_pipeline_step(running) → work → set_pipeline_step(done, summary, ms)
```

Each step writes a one-line human summary and its wall-clock ms. Failures are caught, recorded on the workspace (`status:error` + truncated traceback), and the running step is marked `error`. A `_pace()` helper can floor each step's wall time (`DEMO_MIN_STEP_SECONDS`, default 0/off) so the stepper stays followable when cache replay finishes in milliseconds — it never adds time to genuinely fresh runs.

Stage 1 — Ingest

`ingest/parser.py`. PDF via PyMuPDF emits `{page, text}` per real page. DOCX has no fixed pages, so paragraphs and table rows are flowed into ~3000-char **pseudo-pages** so source-page references stay meaningful. Output: `{num_pages, chars, pages[]}` stored on the workspace at upload time (before the pipeline runs).

Stage 2 — Extract (two LLM passes)

`extract/pipeline.py` + `schemas.py` + `ner.py`.

- **Pass A — metadata** over head+tail chunks: title, issuer, sector (constrained to 8 known sectors), summary, budget, all dated deadlines, evaluation criteria + weights, Q&A sections, submission instructions.
- **Pass B — requirements** over every 12k-char chunk: each compliance obligation as one self-contained sentence with mandatory flag, a category (Eligibility/Technical/Financial/...), and the nearest [PAGE n] source page.
- **Merge** → **dedupe** (difflib similarity) → **pydantic-validate** → **regex NER** normalizes dates (ISO) and PKR amounts.

Pydantic schemas (`RFPProfile`, `Requirement`, `EvaluationCriterion`, `Deadline`, `QAItem`, `MatchResult`) are the **single source of truth** — anything the LLM returns that violates the schema is rejected or coerced (e.g. unknown sector → IT Services, requirement text < 10 chars → dropped).

Stage 3 — Match (RAG + compliance judgment)

`rag/retriever.py` + `rag/matcher.py`.

For each requirement: 1. **Hybrid retrieval** over the 50-record capability library: $0.7 \cdot \text{cosine} + 0.3 \cdot \text{keyword_overlap}$, top-k (default 3). At 50 records an in-memory numpy index is the correct call — no vector DB. 2. **LLM judgment** in batches of 8, against both the retrieved capability records (`CAP-xxx`) and the organization-level **company profile** (`CO-PROFILE`, `data/company_profile.json`).

The matcher prompt encodes a **4-type requirement taxonomy** so each requirement is judged by the right standard — this is load-bearing for honest GO/NO-GO behavior:

Type	Examples	Standard
A. Procedural / format	bid validity, sealed envelopes, copies, pre-bid meeting, scoring rules	Satisfied by commitment in the submission → PASS (C0-PROFILE)
B. Verifiable attributes	named certifications, registrations, turnover, staff credentials	Strict match against C0-PROFILE; a cert in <code>certifications_not_held</code> → GAP
C. Experience / track record	"≥2 projects of value X in sector Y"	Met arithmetically by summary-level CAP records; PARTIAL if short
D. Forward delivery obligations	scope, installation, training, O&M, warranties	Credible-commitment test against domain experience

Each result is PASS / PARTIAL / GAP + confidence + one-sentence rationale + `used_cap_ids`. If the LLM is unavailable, a **deterministic similarity-threshold fallback** assigns status from the top evidence score. `compliance_summary()` computes $\text{compliance_pct} = (\text{PASS} + 0.5 \cdot \text{PARTIAL}) / \text{total}$ and collects mandatory gaps.

Stage 4 — Win probability (the ML core)

This is a deliberate **two-stage design** that avoids target leakage. See `winprob/`.

Why two stages: EDA on the 120-bid history (`eda/EDA_REPORT.md`) shows the outcome is near-deterministic in the *awarded evaluation score* (corr +0.86, hard threshold ~70), while every pre-bid feature alone is noise (the published ablation confirms it). But the awarded score is only known *after* evaluation — it cannot be a live input. So:

- **Stage B — score estimator** (`winprob/estimator.py`): estimates the score a live bid would earn from levers the bid manager actually controls pre-submission — compliance coverage vs history mean, sector win-rate prior, budget alignment to the IQR of historically *won* bids, and a per-mandatory-gap penalty. Returns an estimated 0–100 score with a transparent per-component breakdown.
- **Trained outcome model** (`winprob/train.py` + `features.py`): maps `[est_score, log_budget, compliance%, gaps, doc_pages, response_time, sector one-hots]` → $P(\text{win})$. In the training history `est_score` is the *actual* awarded score; at inference it is Stage B's estimate, so the live feature space matches the training space exactly.

Model selection is honest about $n=120$: XGBoost and L2 logistic regression are both evaluated with **identical StratifiedKFold(5)** folds using `cross_val_predict` (out-of-fold), and the higher CV ROC-AUC ships. All metrics — both candidates, the **ablation without score**, the score-alone AUC, the score→win curve, and feature medians — are persisted to `models/metrics.json` and surfaced live on `/validation`.

Explanation & decision (`winprob/predict.py`):

- **SHAP** per-feature contributions (`TreeExplainer` for XGBoost, `LinearExplainer` in scaled space for LR) against a fixed 60-row background sample.
- **Decision rule:** GO if $P(\text{win}) \geq 0.55$ AND no mandatory gaps; `CONDITIONAL_GO` if $P(\text{win}) \geq 0.40$ OR (high score but mandatory gaps); else `NO_GO`.
- **Decision memo:** an LLM writes a ≤180-word GO/NO-GO memo from the structured payload (probability, SHAP top factors, gaps); a deterministic template is the offline fallback.
- **Comparables:** the 5 most similar historical bids (same sector first, then budget distance).

Stage 5 — Draft & export

draft/generator.py: eight proposal sections (Executive Summary → Value Proposition). The **anti-hallucination contract** is enforced in the prompt and the evidence block: the model may only cite CAP-xxx/CO-PROFILE records actually supplied, must cite inline immediately after each claim, and must state honestly where evidence is thin. Sections are independently editable / approvable / regeneratable (with reviewer feedback) via the API.

export/docx_writer.py: python-docx proposal export and a UTF-8-BOM compliance-matrix CSV, both streamed as downloads.

6. Data model

SQLite schema (db/database.py). JSON-typed columns (pipeline, doc, profile, winprob, effort, evidence, citations) are transparently serialized on write and parsed on read.

Table	Key columns	Purpose
workspaces	id, name, status, pipeline(JSON), doc(JSON), profile(JSON), winprob(JSON), effort(JSON), error	One per uploaded tender; holds the full pipeline state
requirements	id, workspace_id→, idx, text, category, mandatory, source_page, status, confidence, rationale, evidence(JSON), overridden	Extracted requirements + match results + HITL overrides
draft_sections	id, workspace_id→, idx, title, content, citations(JSON), status	Per-section proposal draft + approval state
capabilities	cap_id(PK), domain, summary, certification, year_completed, contract_value(+_m), duration_months, client_type	The 50-record RAG evidence base

Connections are opened per call with `PRAGMA foreign_keys = ON` (cascade deletes), which keeps the layer thread-safe under FastAPI background tasks without an ORM.

Seed data (backend/data/)

Asset	Shape	Role
bid_history.csv	120 × 12	Trains the win model; drives EDA and sector/budget priors
capability_library.csv	50 × 8	Embedded into the RAG index; the only evidence the drafter may cite
company_profile.json	structured fact sheet	CO-PROFILE for org-level (Type B) judgments — certs held vs not held, turnover, staff, registrations
criteria_taxonomy.json	16 entries	Evaluation-criteria taxonomy, synthesized (the sample dataset omitted the sheet the problem statement referenced)

db/seed.py loads the CSVs/JSON into SQLite and triggers the embedding-index build;

data/convert_xlsx.py is the one-time Excel→CSV converter.

7. API surface

All under /api, all returning the {ok, data?} envelope.

Method	Path	Purpose
GET	/health	Liveness
GET	/workspaces	List workspaces
POST	/workspaces	Upload PDF/DOCX → parse → create workspace
GET	/workspaces/{id}	Full workspace detail (profile, requirements, sections, winprob)
DELETE	/workspaces/{id}	Delete (cascades)
POST	/workspaces/{id}/run	Kick off the 4-stage pipeline (background)
PATCH	/requirements/{id}	Human override of a match status (PASS/PARTIAL/GAP)
PATCH	/sections/{id}	Edit content / approve a draft section
POST	/sections/{id}/regenerate	Regenerate a section with reviewer feedback
GET	/workspaces/{id}/export/docx	Download the proposal DOCX
GET	/workspaces/{id}/export/compliance.csv	Download the compliance matrix CSV
GET	/validation	Model metrics + ablation + test report + dataset counts

Interactive OpenAPI docs are auto-served at <http://localhost:8000/docs>.

8. Validation & testing

- **pytest suite** (backend/tests/): schema validation, NER normalizers, retrieval sanity (a known query must return the expected CAP record), ingest+draft, and a **model-metrics gate**. The captured report (models/test_report.json) is surfaced live on /validation.
- **Out-of-fold metrics only.** Every number on /validation comes from cross_val_predict — no row is scored by a model that trained on it — which is why a near-perfect CV AUC reflects a real, almost-trivial rule (the score threshold) rather than memorization.
- **Published ablation.** Removing est_score collapses AUC to $\approx$ coin-flip; this is shown, not hidden, as the evidence that pre-bid features alone carry no signal and that Stage B is necessary.
- **In-app EDA.** Five branded figures (/eda/*.png, generated by eda/generate_eda.py) render on /validation as the evidence behind the model design.

9. Configuration & runtime

Environment (backend/.env, loaded by core/config.py):

Var	Default	Role
ANTHROPIC_API_KEY	—	Claude fallback
OPENAI_API_KEY	—	Primary LLM + embeddings
CLAUDE_MODEL	claude-sonnet-4-6	Fallback model id

Var	Default	Role
OPENAI_MODEL	gpt-5.2	Primary model id
BACKEND_URL	http://localhost:8000	Frontend rewrite target
DEMO_MIN_STEP_SECONDS	0	Floor each pipeline step's wall time for stage demos

On startup the FastAPI lifespan hook self-heals: it creates the DB, seeds capabilities if empty, and trains the win model if `winprob_model.joblib` is missing — so a cold clone is one command from a working demo. Generated artifacts live in `backend/models/` (model, index, metrics, SHAP background) and `backend/cache/` (warmed LLM/embedding responses, committed for offline replay).

Run:

```
# backend (Python 3.11+)
cd backend && pip install -r requirements.txt && uvicorn main:app --port 8000
# frontend (Node 20+)
cd frontend && npm install && npm run dev    # http://localhost:3000
```

10. Directory layout

```
projects/bid-engine/
├── backend/
│   ├── main.py           FastAPI app + lifespan self-heal
│   ├── core/             config.py, llm.py (dual provider + cache + embeddings)
│   ├── api/              routes.py (REST), runner.py (pipeline orchestrator)
│   ├── ingest/           parser.py (PDF/DOCX → pages)
│   ├── extract/          pipeline.py, schemas.py (pydantic SoT), ner.py
│   ├── rag/              embeddings.py, retriever.py (hybrid), matcher.py (4-type)
│   ├── winprob/          features.py, estimator.py (Stage B), train.py, predict.py
│   ├── draft/            generator.py (evidence-grounded sections)
│   ├── export/           docx_writer.py (DOCX + CSV)
│   ├── db/              database.py (SQLite), seed.py
│   ├── data/            bid_history.csv, capability_library.csv, company_profile.json, criteria_taxonomy.json
│   ├── tests/           pytest suite
│   ├── models/          generated: model.joblib, index.npz, metrics.json
│   ├── cache/           warmed LLM + embedding responses (offline replay)
├── frontend/
│   ├── src/app/          page.tsx, workspace/[id]/page.tsx, validation/page.tsx, layout.tsx, globals.css, icon.svg
│   ├── src/components/  ui.tsx, winprob-charts.tsx, theme-toggle.tsx
│   ├── src/lib/         api.ts (typed client), types.ts
│   ├── next.config.ts   /api/* → backend rewrite
├── eda/                 generate_eda.py, EDA_REPORT.md, charts/
├── demo-assets/         test RFQ/RFP/Tender PDFs + make_test_docs.py
├── pitch/               project-proposal.md/.pdf, deck, demo script
└── references/          this doc, workflow-and-tabs.md
```

11. Key design decisions & trade-offs

Decision	Rationale	Trade-off accepted
Two-stage win model (estimator → classifier)	The only strong signal (awarded score) leaks; estimating it pre-bid is the honest fix	Live predictions inherit the estimator's uncertainty
OpenAI-primary, Claude-fallback, disk cache	Venue Wi-Fi is unreliable; a warmed cache replays fully offline	Cache must be pre-warmed against demo docs; committed to the repo

Decision	Rationale	Trade-off accepted
In-memory hybrid retrieval, no vector DB	50 records — a numpy index is faster and simpler than any DB	Won't scale to 100k records without swapping in a real index
SQLite, per-call connections, no ORM	Zero-setup, thread-safe for background tasks, trivial to inspect	Single-writer; fine for single-tenant prototype
Pydantic schemas as single source of truth	Validates every LLM output; unknown/short/invalid data is coerced or dropped	Strictness can drop borderline-valid extractions
4-type matcher taxonomy in the prompt	Lets the system PASS a credible delivery commitment yet GAP a missing certification — honest GO/NO-GO	Prompt complexity; depends on the LLM following the typing
Dual embedding space (openai + hash)	Retrieval survives with no network	Hash embeddings are weaker than learned ones offline

12. Scalability path

The prototype runs on a laptop at a few cents of LLM/embedding spend per bid. The natural product evolution:

1. **Now:** single-company workspace, in-memory retrieval, SQLite, three demo verticals. 2. **Next:** swap the numpy index for a managed vector store as the capability library grows; a capability-library builder that ingests past proposals; a win/loss feedback loop that retrains on each closed bid. 3. **Year one:** multi-tenant SaaS — Postgres in place of SQLite, per-tenant capability libraries, team workflows, and pricing intelligence from historical award data.

Maintainer note: every claim here is grounded in the committed code paths cited inline. When the implementation changes, update this doc and README.md together.